MEAN

[2]:
```
marks=[45,50,55,60,90]
mean=sum(marks)/len(marks)
print("Mean:",mean)  #the mean is calculated by all values and dividing by the total no of values.
```

```
Mean: 60.0
```

MEDIAN

[5]:
```
#It is used o fing the middle value
#Arrange data in ascending order
#Find the middle value

import statistics
marks=[45,50,55,60,90]
median=statistics.median(marks)
print("Median:",median)
```

```
Median: 55
```

[ ]:
```
#When to use median:
#When data contains outliers
#Example:
#Salary=[20000,22000,25000,28000,500000]
#Mean becomes very large because of 5
#Median remains
#25000
#So median gives a better representation
```

MODE

[ ]:
```
#The mode is the value that appears most freqently.
#Example
#data=[10,20,20,30,40]
#Here:
#20 appears 2 times
#So Mode =20
```

[6]:
```
import statistics
data=[10,20,20,30,40]
mode=statistics.mode(data)
print("Mode:",mode)
```

```
Mode: 20
```

[ ]:
```
#When to use mode:
#Finding the most common item
#Examples:
#Most common shirt size
```

[7]:
```
import pandas as pd
df=pd.DataFrame({
    'Marks':[45,50,55,60,90]
})
print("Mean:")
print(df['Marks'].mean())
print("\nMedian:")
print(df['Marks'].median())
print("\nMode:")
print(df['Marks'].mode())
```

```
Mean:
60.0
```

```
Median:
55.0

Mode:
0    45
1    50
2    55
3    60
4    90
Name: Marks, dtype: int64
```

VARIANCE

[ ]:
```
#Variance and Standard Deviation-Measure of Speed
#Measures of spread tell us how far the data values are spread from the centre(mean)
#Two datasets can have the same mean but very different spreads
#Example:
#A=[48,49,50,51,52]
#B=[20,35,50,65,80]
#Both have mean=50
#But dataset b has much more spread out.
```

[ ]:
```
#1.Variance
#Variance measures how far the data values are from the mean on average.
```

[8]:
```
import numpy as np
data=[10,12,14,16,18]
print(np.mean(data))
variance=np.var(data)
print("Variance:",variance)
```

```
14.0
Variance: 8.0
```

STANDARD DEVIATION

[ ]:
```
#Standard Deviation
#50 is the square root of variance

#Interpretation
#Small SD : Data points are closen to mean.
#Large SD : Data points are far from the mean.
```

[10]:
```
#STD CODE
import numpy as np
data=[10,12,14,16,18]
print(np.mean(data))
variance=np.var(data)
std=np.std(data)
print("Variance:",variance)
print("std:",std)
```

```
14.0
Variance: 8.0
std: 2.8284271247461903
```

INTERQUATILE RANGE (IQR)

[ ]:
```
#Interquartile Range(IQR)
#IQR measures the spread of middle 50% data
#It is less affected by outliers that variance and SD

#Formula :
#IQR=Q3-Q1
```

```
#IQR=Q3-Q1

#Where :
#Q1=First Quartile(25%)
#Q3=Third Quartile(75%)

#Example:
#data=[5,10,15,20,25,30,35,40]
#Quartiles
#Q1=13.75
#Q3=31.25

#IQR=Q3-Q1
#17.5
```

[11]:
```
import numpy as np
data=[5,10,15,20,25,30,35,40]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)
```

```
Q1: 13.75
Q3: 31.25
IQR: 17.5
```

[ ]:
```
#IQR FOR Outer Detection
#Outliers are often detecting using
#Lower Limit: Q1-1.5 * IQR
#Upper Limit: Q3+1.5 * IQR
#Any value outside these limits is consisdered an outliers
```

Code JupyterLab Python [conda env:base] *

[14]:
```
#Code
import numpy as np
data=[10,12,14,15,16,18,100]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
lower=Q1-1.5 * IQR
upper=Q3+1.5 * IQR
print(lower)
print(upper)
outliers=[x for x in data if x<lower or x>upper]
print("Outliers:",outliers)
```

```
7.0
23.0
Outliers: [100]
```

PROBABILITY BASIC

[ ]:
```
#Probaility Basics :
#Probability is the measure of how likely an event is to occur
#0=Impossible event
#1=Certein event

#Example:
#Probability of getting haed in a coin toss=0.5
#Probability of rolling a 6 on a die=1/6

#1. Sample space(S)
#The set of all possible outcomes of an experiment

#2. Event(A)
#An event is a subset of a sample space
```

```
#3. Probability of an event P(A)

#Example:
#Find Probability of getting an even number from a die.
#Sample_space=[1,2,3,4,5,6]
#event_A=[2,4,6]
#P_A=len(event_A)/len(Sample_space)
#print("P(A)=',P_A)

#4. Conditional Probability P(A|B)
#Probability of A occuring given that B has already occured

#5. Bayes' Theorem
#Used to calculate the probability of an event based on the prior knowledge

#Example: Diseases Testing
#Suppose:
#P_Disease=0.01
#P_Positive_given_disease=0.95
#P_Positive=0.05
```

[18]:

```
sample_space={1,2,3,4,5,6}
event_A={2,4,6}
print("Sample Space(S):", sample_space)
print("Event A:",event_A)

#2. Proability P(A)

P_A=len(event_A) / len(sample_space)
print("\nProbability of Event A")
print("P(A) =", P_A)
```

```
#3.Conditional proability
#(P|B)

event_B={4,5,6}
intersection=event_A.intersection(event_B)
P_A_given_B=len(intersection)/len(event_B)
print("\nEvent B:", event_B)
print("A ∩ B:",intersection)
print("P(A|B)=", P_A_given_B)
```

```
Sample Space(S): {1, 2, 3, 4, 5, 6}
Event A: {2, 4, 6}

Probability of Event A
P(A) = 0.5

Event B: {4, 5, 6}
A ∩ B: {4, 6}
P(A|B)= 0.6666666666666666
```

BAYES' THEOREM

[25]:
```
#code
#4. Bayes' Therorm
#Ex:
#Disese Prevanlance=1%
#Test sensitivity=99%
#Positive test probability=5%

P_Disease=0.01
P_Positive_given_Disease=0.99
P_Positive=0.05
```

```
P_Disease_given_Positive=(
    P_Positive_given_Disease*P_Disease
)/P_Positive
print("/nBayes' Theorem Example")
print("P(Disease) =", P_Disease)
print("P(Positive|Disease) =",P_Positive_given_Disease)
print("P(Positive) =", P_Positive)

print(
    "P(Disease|Positive) =",
    round(P_Disease_given_Positive,4)
)

print(
    "Percentage =",
    round(P_Disease_given_Positive * 100,2),
    "%")
```

```
/nBayes' Theorem Example
P(Disease) = 0.01
P(Positive|Disease) = 0.99
P(Positive) = 0.05
P(Disease|Positive) = 0.198
Percentage = 19.8 %
```

CORRELATION

```
[ ]: #What is Correlation?
#Correlation meaasures the relationship between two variales

#Example:
#Study Hours = marks
```

```
#Perason Correlation
#Measures linear relationship between two numerical values

#when to use :
#numberical data
#linear relationship
#less outliers

#Example:
#hight anad wight
#study hours and marks
#advertising spend and sales
```

[27]:
```
import pandas as pd
df=pd.DataFrame({
    'Hours': [1,2,3,4,5],
    'Marks': [20,40,50,70,90]
})
corr=df['Hours'].corr(df['Marks'])
print("Pearson Correlation:",corr)

#close to +1
#Strong positive relation
```

```
Pearson Correlation: 0.9948497511671097
```

[ ]:
```
#Spearman Correlation
#Measures rank-based relationship

#Use when
#Data is not normally distributes
#Outliers exist
#Relationship is not perfectly linear
```

Spearman Correlation

[28]:
```
import pandas as pd
df=pd.DataFrame({
    'Rank_Study': [1,2,3,4,5],
    'Rank_Marks': [2,1,3,5,4]
})
corr=df['Rank_Study'].corr(
    df['Rank_Marks'],
    method='spearman'
)
print("Spearman Correlation:", corr)
```

```
Spearman Correlation: 0.7999999999999999
```

[29]:
```
#Correlation Matrix
import pandas as pd
df=pd.DataFrame({
    'Hours': [1,2,3,4,5],
    'Marks': [20,40,50,70,90],
    'Sleep': [8,7,6,5,4]
})
print(df.corr())
```

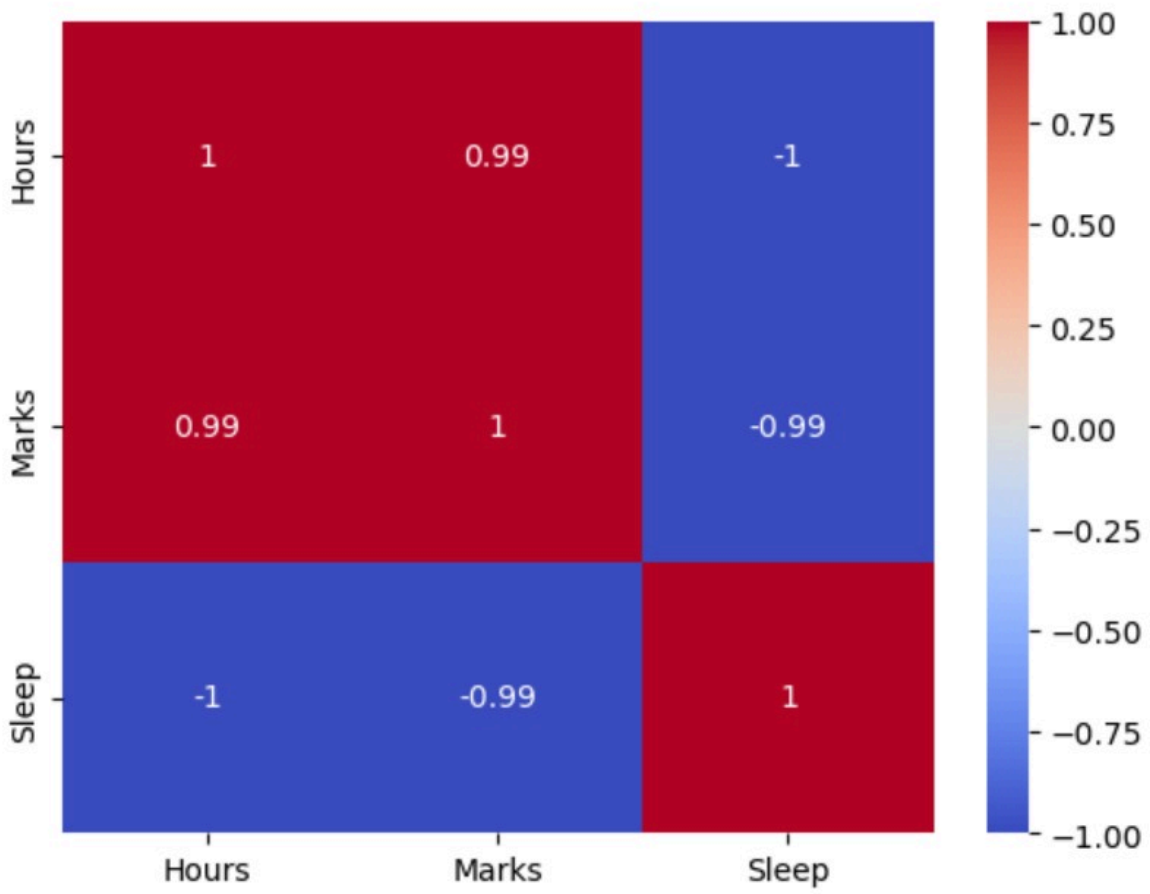
```
         Hours    Marks    Sleep
Hours  1.00000  0.99485 -1.00000
Marks  0.99485  1.00000 -0.99485
Sleep -1.00000 -0.99485  1.00000
```

[ ]:
```
#Interpretation
#hours vs marks=0.99
#strong negative correlation
```

```
Marks  0.99485  1.00000 -0.99485
Sleep -1.00000 -0.99485  1.00000
```

[ ]:
```
#Interpretation
#hours vs marks=0.99
#strong negative correlation
```

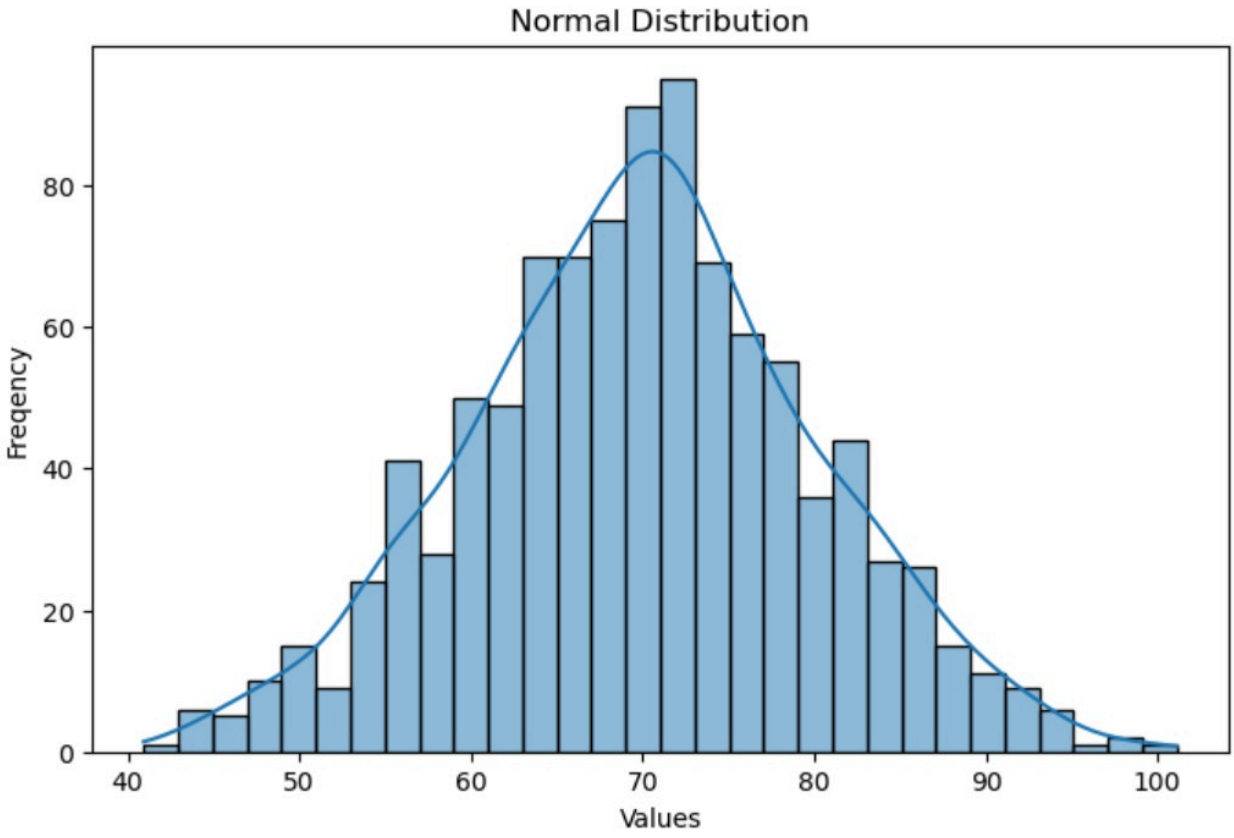
[38]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df=pd.DataFrame({
    'Hours': [1,2,3,4,5],
    'Marks': [20,40,50,70,90],
    'Sleep': [8,7,6,5,4]
})
sns.heatmap(
    df.corr(),
    annot=True,
    cmap='coolwarm'
)
plt.show()

#Common cmap values
#coolwarm
#viribis
#YlGnBu
#blues
#Reds
#greens
#magma
```



[44]:
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data=np.random.normal(
    loc=70,    #mean
    scale=10,  #standard deviation
    size=1000  #number of values
)
```

```
data=np.random.normal(
    loc=70,     #mean
    scale=10,  #standard deviation
    size=1000  #number of values
)
#calculate mean and standard deviation
mean=np.mean(data)
std=np.std(data)

print("Mean:",round(mean,2))
print("Standard Deviation:",round(std,2))

#plot Histogram
plt.figure(figsize=(8,5))
sns.histplot(data,bins=30,kde=True)

plt.title("Normal Distribution")
plt.xlabel("Values")
plt.ylabel("Freqency")

plt.show()
```

```
Mean: 69.9
Standard Deviation: 10.06
```


Normal Distribution
Freqency
Values
0
20
40
60
80
40
50
60
70
80
90
100

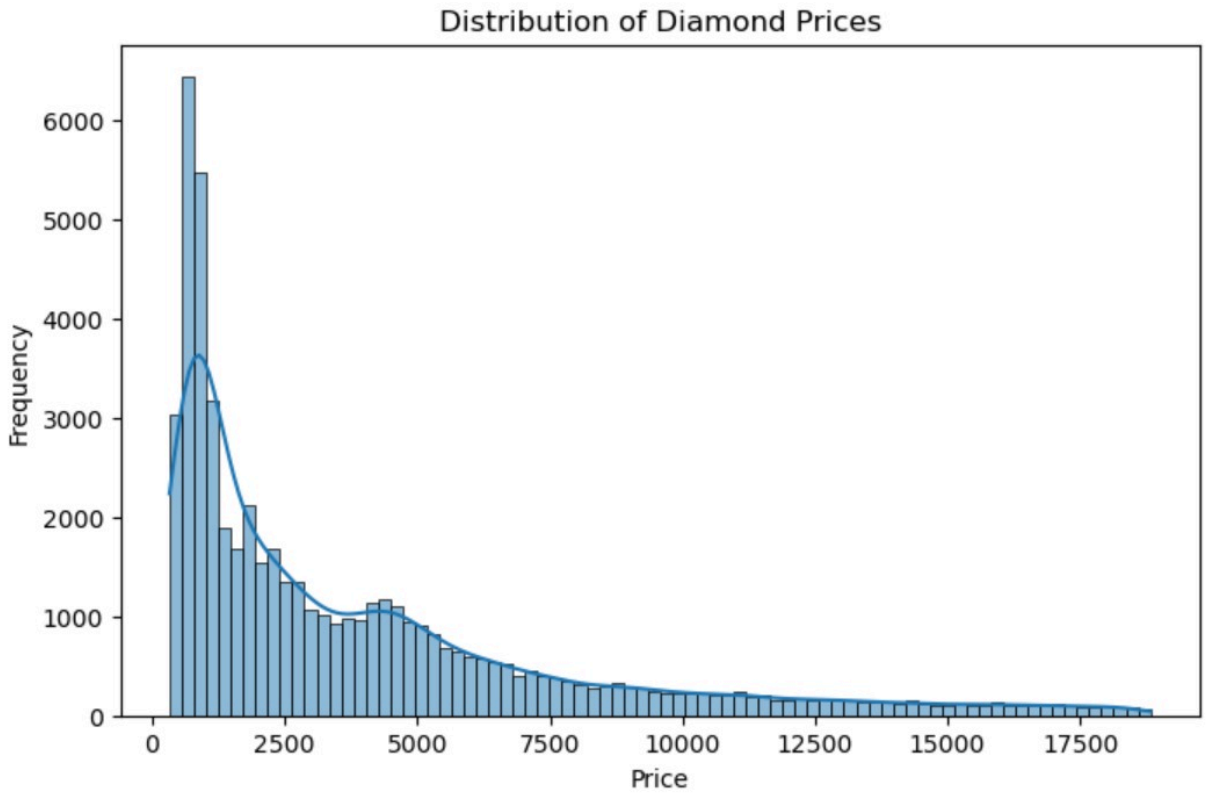
Code JupyterLab Python [conda env:base] *

Values

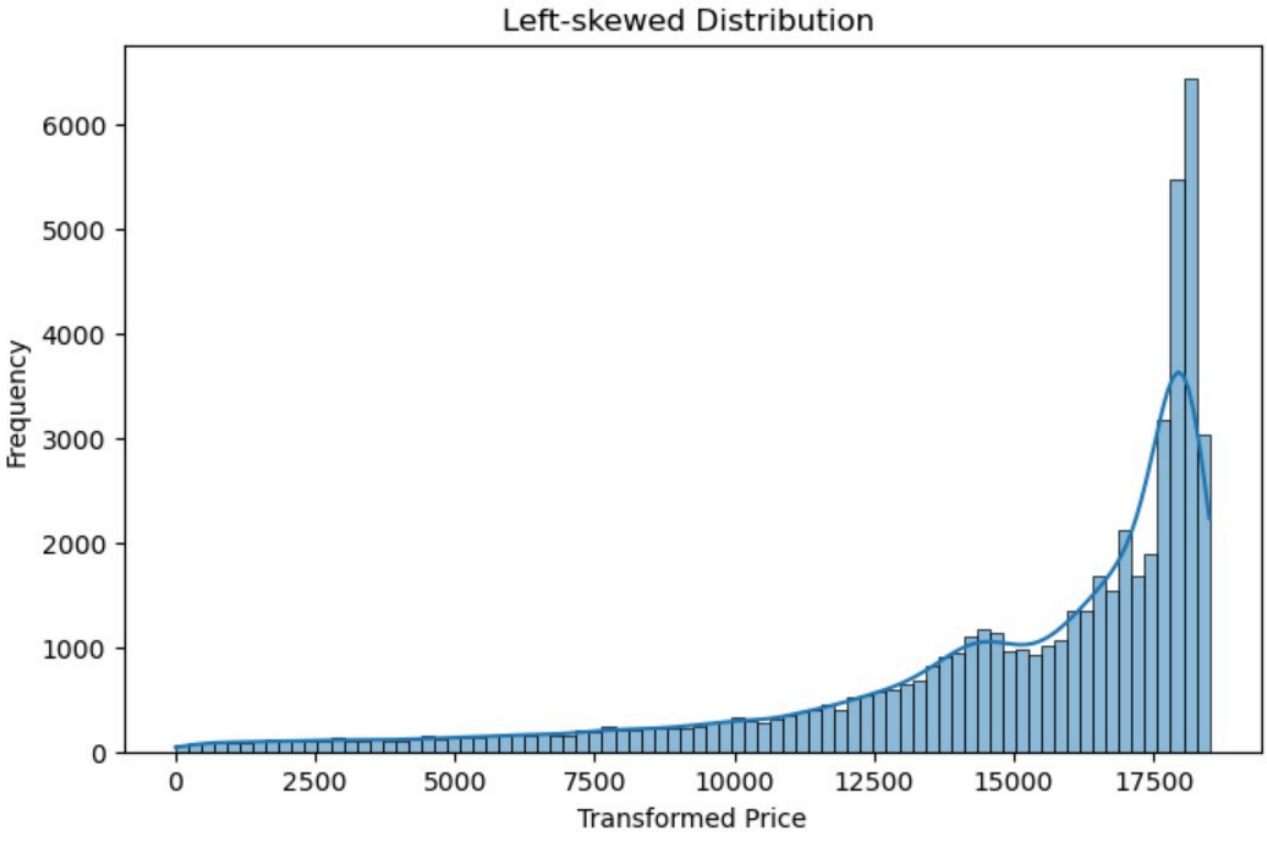
[ ]:
```
#Right-Skewed(Positive Skew)
#Long tail on right
#Mean>median
#Example : income data

#Left skewed(Negative Skew)
#Long tail on the left
#Mean <median
#Example : Easy exam scores
```

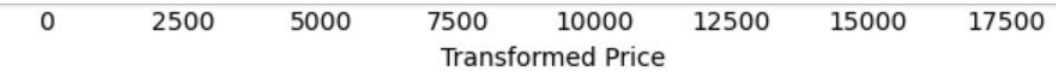
[46]:
```
import seaborn as sns
import matplotlib.pyplot as plt

df=sns.load_dataset("diamonds")
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)

plt.title("Distribution of Diamond Prices")
plt.xlabel("Price")
plt.ylabel("Frequency")

plt.show()
```

```
plt.show()
```


Distribution of Diamond Prices
Frequency
Price
0
1000
2000
3000
4000
5000
6000
0
2500
5000
7500
10000
12500
15000
17500

[48]:
```
import seaborn as sns
import matplotlib.pyplot as plt

df=sns.load_dataset("diamonds")

left_skew=df["price"].max()-df["price"]

plt.figure(figsize=(8,5))
sns.histplot(left_skew,kde=True)

plt.title("Left-skewed Distribution ")
plt.xlabel("Transformed Price")
plt.ylabel("Frequency")

plt.show()
print("Skewness:",left_skew.skew())
```


Left-skewed Distribution
Frequency
Transformed Price
0
1000
2000
3000
4000
5000
6000
0
2500
5000
7500
10000
12500
15000
17500




```
Skewness: -1.6183952833835291
```

[49]:
```
import numpy as np
scores=[60,65,70,75,80]

mean=np.mean(scores)
std=np.std(scores)

score=80

z_score=(score-mean)/std

print("Mean:",mean)
print("Standard Deviation :",std)
print("Z-Score:",z_score)
```

```
Mean: 70.0
Standard Deviation : 7.0710678118654755
Z-Score: 1.414213562373095
```